

Heuristic Algorithm for Solving Travelling Salesman Problem: Adding the Endpoints of the Longest Edges

Fidan Nuriyeva
Department of Computer Science
Dokuz Eylul University
Izmir, Turkey
Laboratory of recognition,
identification and methods of optimal
solutions
Institute of Control Systems
Baku, Azerbaijan
0000-0001-5431-8506

Abstract—This study proposes, a new heuristic algorithm based on insertion heuristic to solve the symmetric Travelling Salesman Problem. In the proposed algorithm, firstly the two longest edges are selected and the starting tour passing through their endpoints is constructed, then the endpoints of the next longest edges are added to this tour and the solution passing through all the vertices is found. This problem is known to be NP-Hard. The proposed method was successfully tested on different-sized problems. The experimental results show that the proposed algorithm outperforms many existing methods.

Keywords—travelling salesman problem, heuristics, insertion heuristic, endpoints of the longest edges

I. INTRODUCTION

The Traveling Salesman Problem (TSP) is a classic combinatorial optimization problem in the field of computer science, mathematics, and operations research. It can be formally defined as follows: Given a list of cities and the distances between each pair of cities, the objective is to find the shortest possible tour that visits each city exactly once and returns to the starting city [1], [2], [3].

The TSP has several applications. Some of its applications are vehicle routing problems, logistics, planning, scheduling and etc. [4], [5], [6].

This problem, commonly recognized as NP-Hard, cannot be precisely solved within a polynomial time frame. Over time, numerous exact and heuristic algorithms have been devised to address this challenge. Among these heuristic approaches, methods like the nearest neighbor algorithm, greedy approach, and insertion heuristics have been utilized to identify efficient solutions for this problem [1], [7], [8].

The main contribution of this study is the development of a heuristic algorithm based on insertion heuristic that solves the symmetric TSP.

II. PROBLEM DEFINITION

The TSP, is typically described within the context of a complete undirected weighted graph denoted as $G = (V, E)$, where V represents the set of vertices, typically enumerated as $V = \{1, 2, \dots, n\}$, and E denotes the set of edges, which are the connections between vertices, specified as $E = \{(i, j): i, j \in V \text{ and } i \neq j\}$. The objective of this problem is to minimize the total distance traveled while visiting every vertex precisely

once, commencing and concluding the tour at a designated starting point.

The Traveling Salesman Problem (TSP) has multiple variations depending on how city-to-city distances are defined. A TSP is considered symmetric when the distance from city i to city j is the same as the distance from city j to city i for all city pairs (i, j) . In contrast, when this symmetry property is not satisfied, it is termed an asymmetric TSP. In this study, our emphasis is on solving a symmetric TSP.

In accordance with the formal definition of the Traveling Salesman Problem (TSP), its mathematical representation is as follows:

$$\min \sum d_{ij} x_{ij} \quad (1)$$

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n \quad (2)$$

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n \quad (3)$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad 2 \leq |S| \leq n - 2, \quad S \subset \{1, 2, \dots, n\} \quad (4)$$

$$x_{ij} \in \{0, 1\} \quad i, j = 1, 2, \dots, n \quad i \neq j \quad (5)$$

The objective function, (1), aims to minimize the total distance traveled. Meanwhile, (2) enforces the restriction that the salesman can depart from any city i only once, and (3) stipulates that the salesman can enter city j only once. (2) and (3) collectively ensure that the salesman visits each city exactly once but do not explicitly prevent the formation of any loops. Additionally, (4) requires that the salesman does not create any loops within subsets of cities. Here, S signifies the number of elements within the set S [4].

III. SOME HEURISTIC APPROACHES FOR SOLVING TSP

There is a need to find an algorithm that will give a solution in a shorter amount of time. As the TSP is NP-hard problem so there is no known algorithm that will solve it in polynomial time. Many algorithms have been tried for solving the TSP heuristically. A few of these algorithms will be explore in this section [2], [7], [8].

A. Greedy Algorithms.

Greedy Algorithms are a method of finding a feasible solution to the TSP. The algorithm creates a list of all edges in the graph and then orders them from smallest distance to largest distance. It then chooses the edges with smallest distance first, providing they do not create a subtour [4].

B. The Nearest Neighbor Algorithm.

The Nearest Neighbor (NN) Algorithm starts at a random vertex in a graph and build a tour by always moving to the nearest neighbor (among the vertices not yet connected to the tour) of the latest vertex to join the tour. When all vertices have been connected to the tour, join the last vertex to the origin to complete the tour [4].

C. Insertion Heuristic.

Insertion Algorithms start with a TSP subtour and insert one new vertex at each step until we have a valid Hamiltonian path. Then, like NN algorithm, it connects the last vertex to the first to complete the TSP tour [4].

IV. PROPOSED ALGORITHM

Proposed algorithm is an insertion heuristic based algorithm. In proposed algorithm, the edges are sorted in descending order, then starting tour is created by selecting the two edges with largest distance. The endpoints of these edges are the first selected vertices. Then, the unselected endpoints of the next edges are added to this tour respectively.

The steps of the algorithm are as following:

Step 0. Sort the edges by decreasing order.

Step 1. The starting subtour is created by selecting the two longest edges A_1-A_2 and A_3-A_4 . The subtour will be as $A_1-A_3-A_2-A_4-A_1$. Here these two edges can also have common vertices such as A_1-A_2 , A_2-A_3 or A_1-A_2 , A_1-A_3 , or A_1-A_3 , A_2-A_3 . (In that case, the starting subtour becomes as $A_1-A_2-A_3-A_1$).

The endpoints of these edges will be the first pair of vertices that is selected: $A_1(x_1, y_1)$, $A_2(x_2, y_2)$, $A_3(x_3, y_3)$ and $A_4(x_4, y_4)$, (in case of common vertex of course there will be 3 endpoints $A_1(x_1, y_1)$, $A_2(x_2, y_2)$ and $A_3(x_3, y_3)$). Delete the vertices A_1 , A_2 , A_3 and A_4 (A_1 , A_2 and A_3 if they have a common vertex) from the set V .

Step 2. Select the next edge. Both endpoints of this edge may be in the subtour, or one or both of them may not be in the subtour. Find the vertex (vertices) that is not in the subtour: Let this vertex be A_k (if both vertices of the edge are not in the subtour then choose A_k and A_{k+1}).

NOTE: Step 3 and Step 4 are also done for A_k and A_{k+1} if both endpoints of the selected edge are not found in the subtour.

Step 3. Find the edge (A_i, A_m) in the tour with endpoints (A_i) and (A_m) where the difference between the sum of the distances; between the endpoints and the length of the edge (A_i, A_m) is minimized for the city (A_k) to be added.

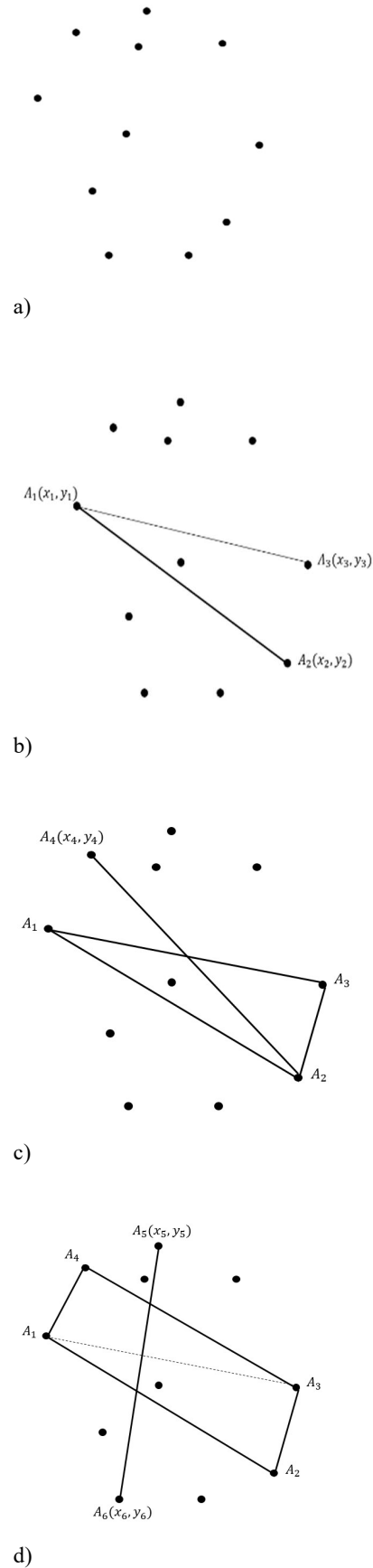
(The edge (A_i, A_m) is deleted from the tour and the edges A_iA_k and A_kA_m are added to the tour. Here, the vertex A_k is selected from the condition $A_iA_k + A_kA_m - A_iA_m = \min$).

Delete the vertex A_k from the set of V ($k = k + 1$).

Step 4. Go to Step 2 until there is no more cities (in other words, $k > n$) to add.

Step 5. Stop.

The steps of the algorithm are shown in the Figure 1 given below.



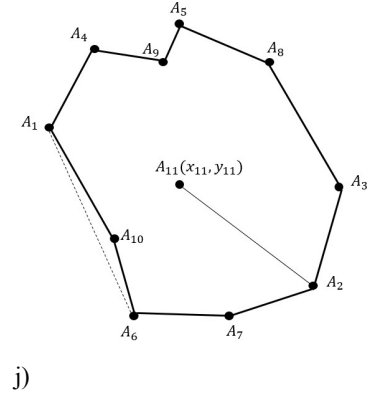
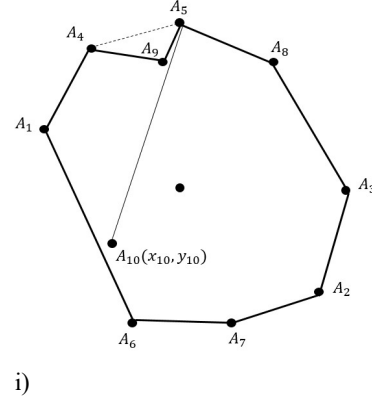
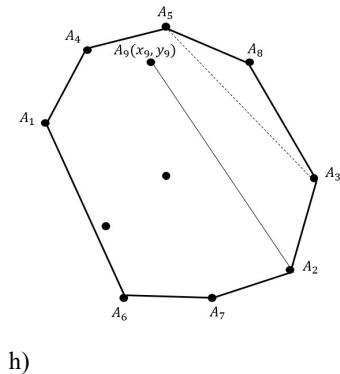
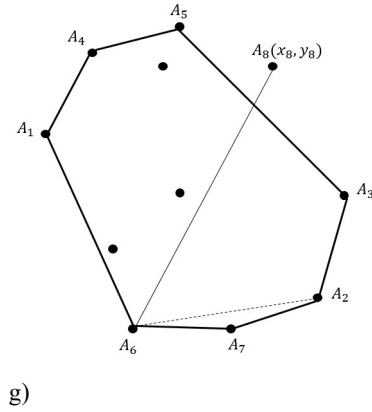
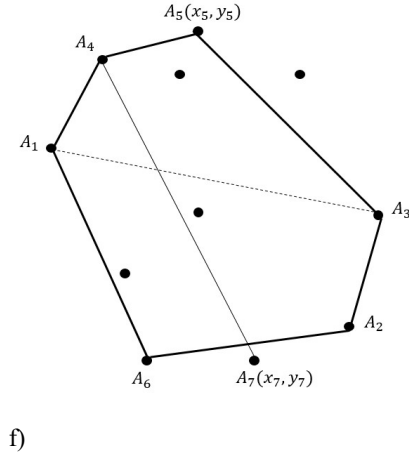
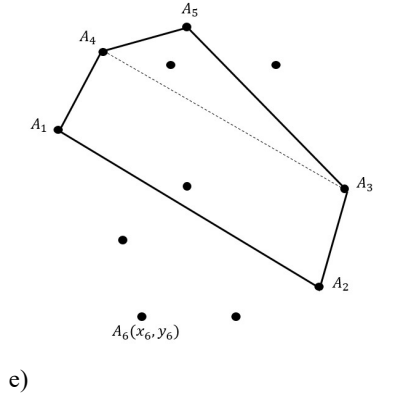


Fig. 1. Demonstration of the steps of the proposed algorithm. a). The set of the vertices (11 vertices) of the graph is given. b). The longest edge ($A_1 - A_2$) is selected and its endpoints (A_1 and A_2) are determined, then the second longest edge ($A_1 - A_3$) is selected and three endpoints (A_3) that will form the starting tour are determined. c). The tour ($A_1 - A_2 - A_3$) passing through the selected endpoints is created and the third longest edge ($A_2 - A_4$) is selected to determine the fourth vertex (A_4) to be added to this tour. d). The tour created in d) is expanded by adding the fourth selected vertex ($A_1 - A_2 - A_3 - A_4$) and the fourth longest edge ($A_5 - A_6$) is selected to determine the fifth vertex (A_5) to be added. As can be seen in the figure, none of the endpoints (A_5 and A_6) of the selected edge are on the tour, so these two Peaks (A_5 and A_6) will be added to the formed tour in the next figures, respectively. These operations are done in figures e) and f). Continuing in this way, the longest edges are selected respectively in the following figures (($A_4 - A_7$, figure f), ($A_6 - A_8$, figure g), ($A_2 - A_9$, figure h), ($A_5 - A_{10}$, figure i), ($A_2 - A_{11}$, figure j)) and vertices ($A_7, A_8, A_9, A_{10}, A_{11}$) to be added to the tour are determined. The last vertex to be added to the tour created in figure j is determined (A_{11}) and in figure k, this vertex is added to the tour and the final tour is given in figure l ($A_1 - A_4 - A_9 - A_5 - A_8 - A_3 - A_2 - A_7 - A_6 - A_{10} - A_{11} - A_1$).

V. COMPUTATIONAL EXPERIMENTS

In this section, we present the outcomes of the computational experiments carried out to assess the performance of our proposed heuristic algorithm. The algorithm was implemented in the C programming language and tested on a system equipped with an Intel(R) Core(TM) i7-4500U CPU operating at 1.80GHz, complemented by 8.00 GB of RAM, running a 64-bit Windows operating system.

The assessment involved applying the algorithm to symmetric Traveling Salesman Problem (TSP) instances gathered from the TSPLIB [10]. The size of each problem is denoted by the numbers attached to their names. In Table I, the tour lengths obtained from the Nearest Neighbor (NN), Greedy heuristics, and our proposed algorithm are presented.

Notably, the optimal results achieved by the algorithms are highlighted in gray cells, while the second-best outcomes are denoted with blue coloring.

TABLE I. COMPUTATIONAL EXPERIMENTS

G	Optimal	NN Time(s)	Greedy Time(s)	Proposed Algorithm
ulysses16	74.108	78.127 0.000	88.923 0.010	75.469 0.000
ulysses22	75.665	86.906 0.000	89.436 0.010	76.951 0.000
eil51	426	505.774 0.016	481.518 0.125	489.802 0.000
berlin52	7542	8182.192 0.000	9954.062 0.281	7782.984 0.000
st70	675	761.689 0.000	746.044 0.485	746.677 0.000
eil76	538	612.656 0.016	617.131 0.672	658.646 0.000
rat99	1211	1369.535 0.016	1528.308 1.875	1546.476 0.000
kroA100	21282	24698.497 0.016	24197.285 1.937	25618.572 0.000
kroB100	22141	25882.973 0.016	25815.214 2.469	25771.402 0.000
kroC100	20749	23566.403 0.015	25313.671 2.610	23389.107 0.015
kroD100	21294	24855.799 0.016	24631.533 2.359	24311.498 0.015
kroE100	22068	24907.022 0.016	24420.355 2.609	28654.093 0.000
rd100	7910	9427.333 0.015	8702.605 2.922	8854.574 0.015
eil101	629	736.368 0.015	789.112 2.609	789.700 0.000
lin105	14379	16939.441 0.015	16479.785 3.187	15540.169 0.000
pr107	44303	46678.154 0.016	48261.816 2.109	46174.964 0.015
ch130	6110	7198.741 0.016	7142.045 7.688	6973.382 0.015
kroA150	26524	31482.020 0.047	31442.994 11.094	30734.513 0.015
kroB150	26130	31320.340 0.047	31519.083 11.156	30101.802 0.015
rat195	2323	2628.561 0.109	2957.176 29.719	2818.824 0.046
kroA200	29368	34547.691 0.125	37650.812 45	37189.000 0.016

As it is seen from the Table 1, computational experiment were made on 21 different-sized problems. In 11 of these problems, algorithm get the best results and 5 of these problems proposed algorithm get second best results. The comparison results demonstrated that the proposed algorithm outperforms other algorithms in term of execution time.

VI. CONCLUSION

In this study, we introduce a novel heuristic algorithm to solve the symmetric Traveling Salesman Problem. Algorithm based on insertion heuristic approach. Moreover, we carried out a sequence of experiments using benchmark instances sourced from the TSPLIB dataset, and conducted a comprehensive evaluation, comparing our findings with results obtained by numerous well known techniques. The outcomes of our experiments demonstrated the impressive performance of our proposed heuristic, showcasing its efficiency across a range of tested instances. It is worth noting that our heuristic outperforms several established methods in this context.

REFERENCES

- [1] Applegate, D. L., Bixby, R. E., Chavatal, V. and Cook, W. J., 2006, The Travelling Salesman Problem, A Computational Study, Princeton University Press, Princeton and Oxford, 593p.
- [2] Johnson, D. S. and McGeoch, L. A., 1997, The Traveling Salesman Problem: A Case Study, Local Search in Combinatorial Optimization, John Wiley & Sons, 215-310 pp.
- [3] Lawler, E. L., Lenstra, J. K., Rinnoy Kan A. H. G. and Shmoys, D. B., 1985, The Travelling Salesman Problem: A Guided Tour of Combinatorial Optimization, John Wiley & Sons, 465p.
- [4] Davendra, D., 2010, Travelling Salesman Problem, Theory and Applications, InTech.org, 324p.
- [5] Punnen, A., 2002, The Traveling Salesman Problem: Applications, Formulations and Variations, In Gutin and Punnen, chapter 1, Kluwer Academic Publishers, Netherlands, 1-28 pp.
- [6] Reinelt, G., 1994, The Traveling Salesman: Computational Solutions for TSP Applications, Springer-Verlag, Germany.
- [7] Johnson, D. and Papadimitriou, C., 1985, Performance guarantees for heuristics, In Lawler et al, The Traveling Salesman Problem, A Guide for Tour of Combinatorial Optimization, chapter 5, 145-180 pp.
- [8] Matai, R., Singh, S., and Mittal, M. L., 2010, Traveling Salesman Problem: an Overview of Applications, Formulations, and Solution Approaches, Traveling Salesman Problem, Theory and Applications, InTech.
- [9] Gutin, G. and Punnen, A. (eds.), 2002, The Traveling Salesman Problem and Its Variations, volume 12 of Combinatorial Optimization. Kluwer, Dordrecht, 830p.
- [10] "TSPLIB." comopt.ifi.uni-heidelberg.de. <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/> (accessed May 31, 2023).